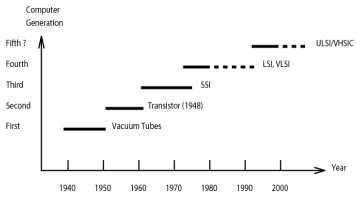
cs230

Support Slides
Winter 2017

A Little History



First (1938-1953): Electronic Numerical Integrator & Computer (ENIAC)
Electronic Discrete Variable Automatic Computer (EDVAC) ←1st Stored Program
Assembly Language, single user, fixed point arithmetic, CPU assisted I/O

Second (1955-1964): TRADIC (Bell Labs), Stretch (IBM 7030, inst. lookahead & error correction), IBM 7090, CDC 1604, Univac LARC.
HLL: FORTRAN (1956), Cobol (1959), Algol (1960), Compilers, Libraries, Batch, Monitor, Floating Point, I/O Processors, Multiplexed Memory Access

Third (1965-1974): IBM 360/370, CDC 6600, TI ASC, PDP-8, ILLIAC 4 (1968) ←(8x8 Mesh Connected Parallel Computer)
Microprogramming, Cache, Multiprogramming, Times-shared OS, Intelligent Compilers, Virtual Memory

Fourth (1975-1990): VAX 9000, Cray X-MP, IBM 3090, BBN TC2000, MPP (Goodyear/NASA)
HLL for Scalar & Vector Data, Vectorizing Compilers, Languages and Environments for Parallel Processing

Adapted from K. Hwang, *Advanced Computer Architecture: Parallelism, Scalability, Programmability*, McGraw-Hill, Inc., Reading, 1993.

… and the FIFTH Generation ?

… and the FIFTH Generation ?

- 1980's …
  - Artificial Intelligence becomes new hype …

… and the FIFTH Generation ?

- 1980's …
  - Artificial Intelligence becomes new hype …
  - Japan calls the Fifth Generation AI plus Parallel and Distributed Computing:
    Lisp ⇨ Prolog ⇨ Concurrent Prolog

… and the FIFTH Generation ?

- 1980's …
  - Artificial Intelligence becomes new hype …
  - Japan calls the Fifth Generation AI plus Parallel and Distributed Computing:
    Lisp ⇨ Prolog ⇨ Concurrent Prolog
  - USA calls for Tera-Flop Computing by mid 2000's
    - High Performance Computing Initiative (HPCI)

High Performance Computing Initiative (HPCI)

- Use applications to justify/demonstrate sustained Tera-Flop performance.
  - Actual problems that require numerical computer solutions and whose time complexity is too long for *current* super-computers
  - Numerous scientific and engineering applications
  - Modeling, simulation, and analysis of complex systems: e.g. climate, galaxies, molecular structures, nuclear explosions, etc.
  - Business and Internet applications (although Internet did not exist yet)
    - E-commerce – ex. Amazon
    - Web servers – ex. Yahoo, Google
  - Many more…

Grand Challenge Problems

- "*fundamental problems in science and engineering that have broad economic and/or scientific impact and whose solution can be advanced by applying high performance computing techniques and resources*"

- Originally posed by the High Performance Computing and Communications program of US Govt., many problems added by committees/agencies since then.

  - **Example: The Human Genome – A great success !!!**

Other examples

- Ground water remediation
- Simulation of X-ray clusters – study of galaxy formation
- Design and simulation of aerospace vehicles
- Climate modeling
- Improving environmental decision making
- Discovery of non-renewable energy sources
- Understanding bio-molecular structures
- For more details on these and other problems
  - http://www.nitrd.gov/pubs/200311_grand_challenges.pdf
  - http://ceee.rice.edu/Books/CS/chapter1/intro52.html

June 1989, Workshop at NASA-GSFC

- Objective: Agree on a number of Grand Challenge Problems to show sustained Tera-Flop performance.

June 1989, Workshop at NASA-GSFC

- Objective: Agree on a number of Grand Challenge Problems to show sustained Tera-Flop performance.
- Important participant requests that another "C" be added to HPC, to create the HPCC initiative:
  - Additional "C" stands for "Communications"

June 1989, Workshop at NASA-GSFC

- Objective: Agree on a number of Grand Challenge Problems to show sustained Tera-Flop performance.
- Important participant requests that another "C" be added to HPC, to create the HPCC initiative:
  - Additional "C" stands for "Communications"
- Senator Al Gore is responsible for added C … "the Information Superhighway" … and the rest is history !!!

HPCC Grand Challenge Areas

- Aerospace
- Computer Science
- Energy
- Environmental Monitoring and Prediction
- Molecular Biology and Biomedical Imaging
- Product Design and Process Optimization
- Space Science

Aerospace

- **Computational Aero-sciences Project**
  NASA - NASA Ames, NASA Langley and NASA Lewis
  - Accelerate the development and availability of high-performance computing technology that will be of use to the U.S. aerospace community, facilitate the adoption and use of this technology by the U.S. aerospace industry, and hasten the emergence of a viable commercial market for hardware and software vendors to exploit this lead.

- **High performance computational methods for coupled field problems and GAFD turbulence**
  NSF - Colorado, Minnesota, and the National Center for Atmospheric Research
  - Develop and implement algorithms and software on parallel computers for solving field problems in structural and fluid dynamics and studying highly turbulent flows which arise in geophysical and astrophysical fluid dynamics.

Computer Science

- **High performance computing for learning**
  NSF - MIT, Brown and Harvard
  - Develop, implement, and test new mathematical techniques, software, and hardware for high performance computers with the ultimate goal of getting computers to "see, move, and speak."

- **Parallel I/O methodologies for I/O-intensive Grand Challenge applications**
  NSF - Caltech and Illinois
  - Investigate and develop strategies for the efficient implementation of I/O intensive applications on a specially configured Intel Paragon computer. They will characterize I/O behavior and performance, define I/O models and methodologies, and develop, implement and test tools to support scientific applications with large I/O requirement

Energy

- **Mathematical combustion modeling**
  DOE
  - Developing adaptive parallel algorithms for computational fluid dynamics and applying them to combustion models.

- **Numerical Tokamak project**
  DOE - Lawrence Livermore, Texas, UCLA, Oak Ridge, Princeton, NASA JPL, Cornell, Los Alamos, Caltech, National Energy Research Supercomputer Center
  - Develop and integrate particle and fluid plasma models on massively parallel machines as part of the multidisciplinary study of Tokamak fusion reactors.

Energy (contn'd)

- **Oil reservoir modeling**
  DOE - Texas A&M, Brookhaven, Oak Ridge, Rice, Stony Brook, South Carolina, and Princeton
  - Develop software for massively parallel computers that calculates fluid flow through permeable media. The project has a dual application, focusing on methods that solve modeling problems for petroleum reservoirs and for groundwater contamination.

- **Quantum chromo-dynamics calculations**
  DOE - Los Alamos
  - Developing lattice gauge theory algorithms on massively parallel machines for high energy physics and particle physics applications.

Environmental Monitoring and Prediction

- **Adaptive coordination of predictive models with experimental observations**
  NSF - Stanford and NASA Ames
  - Using a predictive computer model carrying out simulations in real time and a laboratory test bed, the team will investigate the potential for the interplay of the simulations and the experimental facility to estimate what data need to be gathered, as well as the location and resolution of this data, in order that accurate predictions of the future behavior of a complex nonlinear fluid system such as the atmosphere or the ocean can be made.

- **Computational chemistry**
  DOE - Argonne, Pacific Northwest Laboratory, Allied Signal, du Pont, Exxon, and Phillips
  - Develop new parallel algorithms, software, and portable tools for computational chemistry, and develop modeling systems for critical environmental problems and remediation methods.

Environmental Monitoring and Prediction (cont'd)

- **Data analysis and knowledge discovery in geophysical databases**
  NASA - UCLA, NASA JPL
  - Demonstrate the applicability of information systems for geophysical databases to support cooperative research in earth-science projects.

- **Development of algorithms for climate models scalable to TeraFLOP performance**
  NASA - NASA Goddard
  - Develop a high-resolution global climate model capable of centuries-long calculations on massively parallel machines at teraFLOP speed.

Environmental Monitoring and Prediction (cont'd)

- **Development of an Earth system model: atmosphere/ocean dynamics and tracers chemistry**
  NASA - UCLA, Princeton, Berkeley, Santa Barbara, JPL, Lawrence Livermore
  - Develop a model of the coupled global atmosphere-global ocean system, including chemical tracers that are found in, and may be exchanged between the atmosphere and the oceans. Use the model to study the general circulation of the coupled atmosphere-ocean system, the global geochemical carbon cycle, and the global chemistry of the troposphere and stratosphere.

- **A distributed computational system for large scale environmental modeling**
  NSF - Carnegie Mellon and MIT
  - Use high performance heterogeneous computing systems, advanced software environments, parallel architectures, and networks to develop algorithms for multiphase chemistry and aerosol dynamics and a distributed computing approach for simultaneous solution and sensitivity of environmental models.

Environmental Monitoring and Prediction (cont'd)

- **Earthquake ground motion modeling in large basins**
  NSF - Carnegie Mellon, USC and the National University of Mexico
  - Develop new mathematical models and software tools to demonstrate the capability for predicting, by simulation on parallel computers, the ground motion of large basins during strong earthquakes, and use this capability to study the seismic response of the Greater Los Angeles Basin.

- **Four-dimensional data assimilation for massive Earth system data analysis**
  NASA - NASA Goddard, NASA JPL, Syracuse
  - The goal of data assimilation is the calculation of consistent, uniform, spatial and temporal representations of the Earth environment that can be used for scientific analysis and synthesis. This involves the collection of diverse Earth observational data sets, and the incorporation of these data into models of the ocean, land surface, and atmosphere, including chemical processes.

Environmental Monitoring and Prediction (cont'd)

- **Global climate modeling**
  DOE - Los Alamos, Argonne, Oak Ridge
  - Numerical studies of the Earth's climate using general circulation models of the atmosphere and ocean.

- **Groundwater transport and remediation**
  DOE - Texas A&M, Brookhaven, Oak Ridge, Rice, Stony Brook, South Carolina, and Princeton
  - Develop software for massively parallel computers that calculates fluid flow through permeable media. The project has a dual application, focusing on methods that solve modeling problems for petroleum reservoirs and for groundwater contamination.

Environmental Monitoring and Prediction (cont'd)

- **High performance computing for land cover dynamics**
  NSF - Maryland, New Hampshire, Indiana, and NASA Goddard
  - Develop techniques to support access and analysis of remotely sensed data stored on parallel disk systems and use those techniques to facilitate the study of global ecological responses to climate changes and human activity.

- **Massively parallel simulation of large scale, high resolution ecosystem models**
  NSF - Arizona
  - Establish new algorithms and implementations for massively parallel processing that integrate geographical information systems databases with cellular discrete-event methodology to express large scale realistic ecosystem models and visualize their simulated behavior. The focus will be on monitoring and predicting landscape and ecosystem changes for large geographic regions

Molecular Biology and Biomedical Imaging

- **Advanced computational approaches to biomolecular modeling and structure determination**
  NSF - Illinois, Duke, NYU, Yale, and Eli Lilly Corporation
  - Develop models and molecular dynamics algorithms for a widely used program for structural biology (X-PLOR) in order to advance the fundamental understanding of molecular biology and pharmacology.

- **Computational biomolecular design**
  NSF - Houston Use emerging scalable parallel computers and software to develop and implement new methods for solving critical problems in biomolecular design.

Molecular Biology and Biomedical Imaging (cont'd)

- **Computational structural biology**
  DOE - Caltech, Argonne, University of Washington, and UCLA
  - Understanding the components of genomes and developing a parallel programming environment for structural biology.

- **High performance imaging in biological research**
  NSF - Carnegie Mellon and Pittsburgh
  - Use the latest technologies in light microscopy and reagent chemistry with advanced techniques for computerized image analysis, processing and display, implemented on high-performance computers to produce an automated, high speed, interactive tool that will make possible new kinds of basic biological research on living cells and tissues.

Molecular Biology and Biomedical Imaging (cont'd)

- **Understanding human joint mechanics through advanced computational models**
  NSF - Rensselaer Polytechnic and Columbia
  - Develop automated and adaptive three-dimensional finite element analysis and parallel solution strategies to describe nonlinear moving contact problems characteristic of the biomechanics of joints in the human musculoskeletal system using the actual anatomic geometries and the multiphasic properties of the tissues in the joint.

Product Design and Process Optimization

- **First-principles simulation of materials properties**
  DOE - Oak Ridge, Brookhaven, NASA Ames
  - Investigate new methods for performing large-scale, first-principles simulation of materials properties using a hierarchy of increasingly more accurate techniques that exploit the power of massively parallel computing systems.

- **High capacity atomic-level simulations for design of materials modeling**
  NSF - Caltech, Columbia and NASA JPL
  - Formulate and implement new methodologies for parallel computers to carry out high capacity atomic-level simulations for design of materials, and apply the resulting software to critical industrial materials problems.

Space Science

- **Black hole binaries: coalescence and gravitational radiation**
  NSF - Texas, Illinois, Syracuse, Pittsburgh, Penn State, Northwestern, North Carolina and Cornell
  - Create a computational toolkit to provide modular development tools to support the study of coalescence of astrophysical black holes and the gravitational radiation emitted via the numerical solution of Einstein's equations for gravitational fields.

- **Convective turbulence and mixing in astrophysics**
  NASA - Colorado, Michigan State, Chicago, Argonne, NCAR
  - Develop the next generation of multi-dimensional hydrodynamic codes for astrophysical simulations involving turbulent convection, based on the use of massively parallel machines.

Space Science (cont'd)

- **Cosmology and accretion astrophysics**
  NASA - Los Alamos, Syracuse, Penn State, Caltech, Australian National University
  - Develop parallel, scalable particle codes (N-body, smoothed particle hydrodynamic (SPH), and hybrid) based on hierarchical tree data structures and use them to study astrophysical problems.

- **The formation of galaxies and large-scale structure**
  NSF - Princeton, Illinois, Pittsburgh, MIT, Indiana, San Diego
  - Explore different numerical algorithms, mesh adaptation strategies, programming models and new software technologies in order to obtain detailed numerical simulations that can help answer the question: "What is the origin of large-scale structure in the universe and how do galaxies form?"

Space Science (cont'd)

- **Large scale structure and galaxy formation**
  NASA - University of Washington, University of Toronto
  - Develop the tools needed for high performance N-body simulations, and use these to test the "standard model" for the origin of galaxies and large-scale structure by accurately evolving it into its present highly nonlinear state.

- **Radio synthesis imaging**
  NSF - Illinois, Wisconsin, Maryland, Berkeley
  - Implement a prototype of the next generation of astronomical telescope systems - remotely located telescopes connected by high-speed networks to very high performance computers and on-line data archives.

- **Solar activity and heliospheric dynamics**
  NASA - Naval Research Laboratory, NASA Goddard
  - Develop parallel algorithms for solar and heliospheric modeling.

How do we tackle them?

- Faster algorithms
  - Faster sequential approaches to solving the problem
  - Parallel algorithms
- Faster Machines
  - Faster <processors, memory, interconnect>
  - Parallel machines
- Improvements in all these are necessary, but, in most cases *Sequential algorithms running on single processor machines are not enough : need for parallel algorithms on parallel machines*

Why Sequential Architectures are not enough

- Clock speeds are bounded by physical laws
- Instruction level parallelism already exists in processors
  - Pipelining
  - Superscaler processors
  - VLIW (Very Long Instruction Word) Architectures

  But requires very complex hardware and/or sophisticated compilers
- Vector processors work well only for certain types of problems

High Performance Computing Initiative (HPCI)

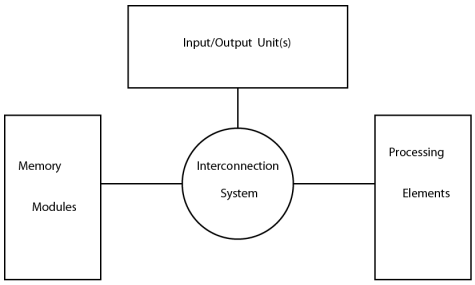
- Thesis: Only Scalable Concurrent Computing may solve the problems.

Parallel and Distributed Computing

Components of a Parallel Machine

- Processor
- Memory
- Interconnect (Processor-to-Processor and Processor-to-Memory)
- I/O
- Programming model and tools

Classification of Parallel Architectures (Flynn's Taxonomy)



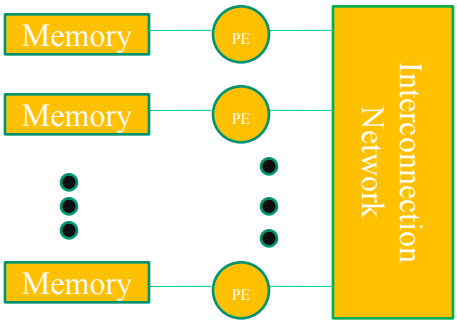
	Single Data Stream SD	Multiple Data Stream MD
Single Instruction Stream SI	SISD: Uniprocessor (VAX, IBM, RISC)	SIMD: Array Processors (MPPs, Associative)
Multiple Instruction Stream MI	Pipeline, Systolic Processors	MIMD: Multiprocessors CM-5, IPSC, Paragon

After Michael Flynn, *Some Computer Organizations and their Effectiveness*, IEEE Transactions on Computers, September 1972.

Parallel/Distributed Architectures - Classification

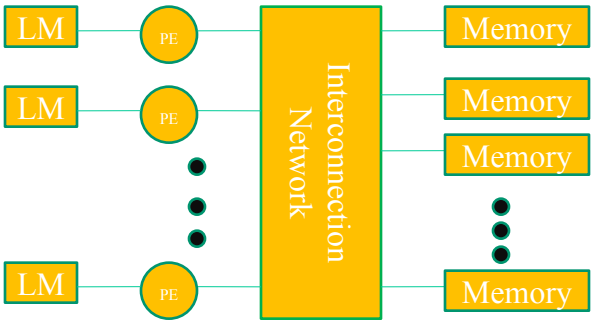
- SISD (Single Instruction, Single Data)
  - conventional single processor machines
  - Not really parallel !
- SIMD (Single Instruction Multiple Data)
  - Multiple processing units (PE) working in lockstep
  - Same instruction executed by all PEs in each step on different data, single control unit to tell PEs what to do
- MISD (Multiple Instruction Single Data)
  - No real machine mapping
- MIMD (Multiple Instruction Multiple Data)
  - Multiple PE, two PE can be executing different instructions at the same time

Basic Parallel/Distributed Architecture



- *SIMD or MIMD Architecture.*
  - *Memory can be distributed (exclusive address space for each PE) or can be made look as shared (single address space for all PE)*

Basic Parallel/Distributed Architecture



- *MIMD Architecture. Some Local Memory and a Modular Shared Memory.*
  - *Local Memory can be shared (single address space for all PE) or distributed (different address space for different PE)*

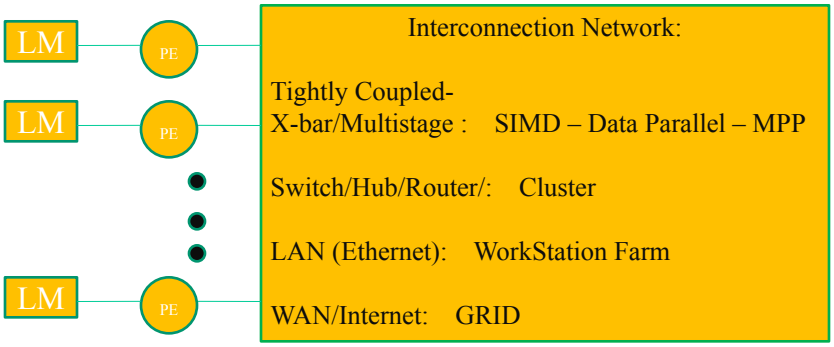
Examples

- SISD
  - mainframes, workstations, PCs.
- SIMD Shared Memory
  - Hitachi S3600 Series
- MIMD Shared Memory
  - Cray J90/T90, DEC Alphaserver, SGI Origin 3000
- SIMD Distributed Memory
  - Cambridge Parallel Processing Gamma II Plus
- MIMD Distributed Memory
  - Cray T3D/T3E, Cray XT3, plus recent workstation clusters (IBM SP2, DEC, Sun, HP).

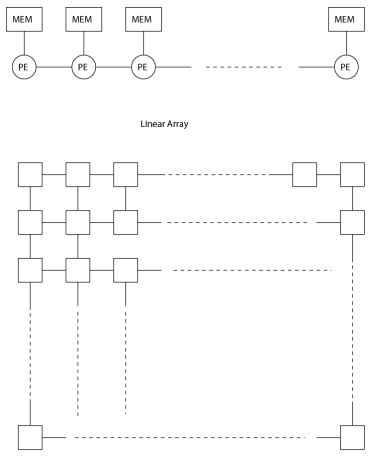
For a good overview of architectural classes for HPC, see
http://www.netlib.org/utk/papers/advanced-computers/

Differences between Architectures

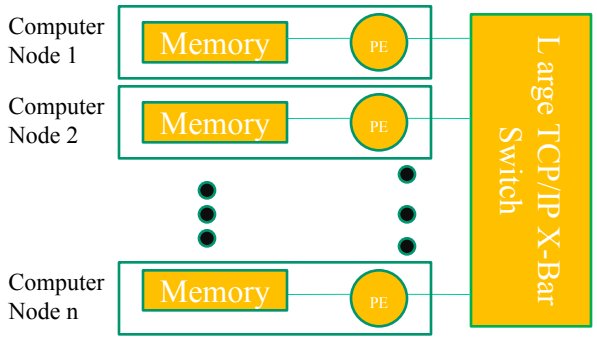
- Interconnection Network Speed (Latency and Bandwidth) differentiates between different concurrent computing architectures:
- Many network details have been oversimplified here … for simplicity's sake …



Two Examples of point-to-point Architectures



Typical Cluster Architecture



Other Nomenclatures Used

- Symmetric Multiprocessing (SMP)
- Massively Parallel Processors (MPP)
- Cache-coherent Non-uniform Memory Access (CC-NUMA)
- Distributed Systems
- Clusters

SMP

- Shared memory MIMD
- Multiple processors of the same type
- Processors and memory connected to the same bus
- All processors are treated as equal, any task can be done by any processors
- Typical no. of processors less than 100 (typically 2-64)
- Task allocation to processors controlled by OS
- Most common OS's like Windows, Linux support SMPs

MPP

- Specialized architectures with large number of processors (hundreds to thousands)
- Specialized fast interconnect switches to connect processors to processors and processors to memory
- Can be SIMD or MIMD, shared or distributed memory
- Examples – Cray T3D, MasPar's MP1/2, Thinking Machines CM1/2, CM5.

NUMA

- Each processor with its own physical memory
- Shared virtual address space
- Accesses to addresses in local memory faster
- Accesses to addresses in remote memory handled by hardware routers, slower
- Local cache at each processor
- Cache-coherency protocol needed to keep caches consistent (CC-NUMA)
- Smaller number of processors (less than 100)
- Examples – SGI Origin, Sequent NUMA-Q

Clusters

- Collection of interconnected **stand-alone** computers (*nodes*) that work together as a **single computing resource.**
- Front-end where tasks are submitted, allocated and load balanced among back-end machines transparently (*Single System Image*).
- Machines usually run same operating system kernel in each node.
- Distributed OS is challenging.
- Number of nodes can be from tens to thousands

Clusters are becoming increasingly popular, why?

Problems with Conventional Supercomputers

- Very costly
  - Specialized processors not cheaply available
  - Specialized interconnects to support bandwidth needed
- Harder to program
  - Uncommon processors
  - Lack of standard programming model and interface
  - Lack of standard tools
- Shorter life span
  - Harder to upgrade
  - Scalability a problem for many

Enablers for Clusters

- Individual machines are becoming very powerful, no need for specialized processors to achieve required speed at each node
- Faster network technology reduces the need for specialized, proprietary interconnects between processors
- Incremental scalability – add nodes as needed
- Use of common-off-the-shelf (COTS) components implies lower cost and ready availability
- Development tools are more mature
- Standardized programming interfaces like PVM, MPI etc. makes programs portable

Popularity of Clusters in HPC

- www.top500.org - list of the top 500 supercomputers in the world, updated twice per year
- Ranked according to their performance on the standard *Linpack* benchmark
- 294 of them in the current list (Nov. 2004) are clusters!
- Highest rank of a cluster – 2 (approx. 51 teraflops)

Cluster Components

A typical cluster

- Stand alone machines
- A fast network connecting them
- Low latency communication protocols
- Software to give ***Single System Image***
- Programming Tools

Additional components:

- Network RAM
- Parallel I/O

Example: Berkeley NOW

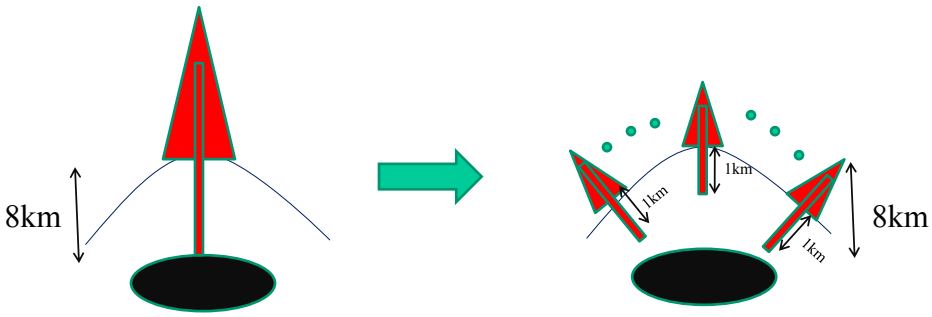
- 100+ SUN UltraSparc machines (Ultra 170)
- 200 disks
- Myrinet interconnection within cluster– 160 MB/s
- Switched Ethernet to ATM backbone for external communication
- GLUnix – global OS over Solaris for process management
- AM (Active Message) communication protocol
- MPI for programming

Cluster Classification

- Target applications
  - High-Performance Clusters – for scientific apps.
  - High-Availability Clusters – for critical apps.
- Node ownership
- Node Hardware
- Node OS
- Node Configuration
- Clustering Levels

How to use a Concurrent Computer

- Concurrent Computing Example



Concurrent Computing

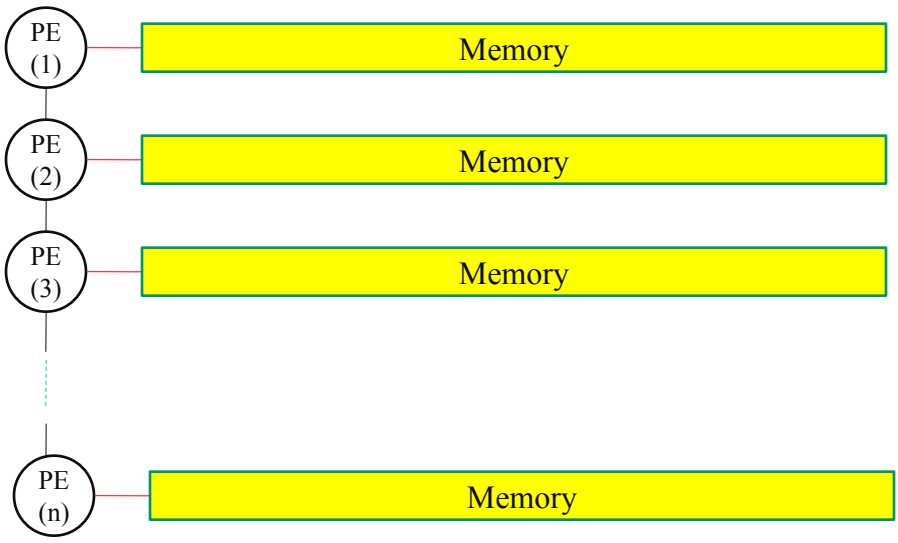
- Break a problem into smaller sub-problems that can be solved concurrently.
- Integrate solution to larger original problem.
  - Typical decomposable problems:
    - Array computations (Vector, Matrix calculations)
    - PDEs, ODEs, Linear Systems Solution.
    - Transforms: Laplace, Fourier, Radon (CAT scanner)

Example: Matrix Multiply

- Given A=[aij] and B=[bij] (i and j from 1 to n) compute

  C = A x B

  In the following architecture:



Matrix Multiply (cont'd)
Basic Matrix Multiply Loop:

```
C(i,j) is initialized to ZERO
For i=1 to n, do {
   For j=1 to n, do {
     For k=1 to n, do {
              C(i,j) = C(i,j) + a(i,k) * b(k,j)
     }
   }
}
```


Matrix Multiply (cont'd)

- Need to do the following:
  - Decide on a storage scheme
  - Decide on a sequence of Computations/Communications performed by each processor
- Roll up your sleeves and let's get to work !

Example: 2D Fast Fourier Transform

- Problem arises in Radar Data Processing
  - Need for speed if time sensitive application such as Air Traffic Control

- FFTslides.pdf

